

This presentation uses the Pretendard and D2Coding font.

.NET Universe 2026

Korea's largest .NET developer conference

Presented by

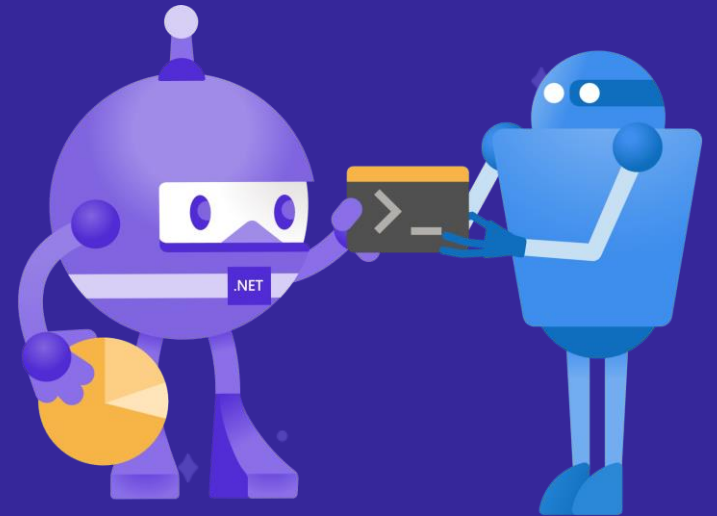


**.Net
Dev**

AWS Transform for .NET: 레거시 .NET Framework에서 크로스 플랫폼 .NET으로 현대화

강병천

GTM Specialist SA, AWS



Agenda



- .NET 현대화의 필요성
- AWS Transform for .NET 동작원리
- 데모
- 변환 이후

레거시 .NET 애플리케이션의 현대화 필요성

왜 Windows 기반
워크로드를 클라우드
네이티브, 오픈소스로
현대화를 해야 하는가?



애플리케이션 계층



데이터베이스 계층

배포

.NET Framework 3.5+

SQL Server

Windows Server

비용 부담

레거시 Windows/SQL Server
환경은 라이선스 비용, 지원 비용,
업그레이드 비용이 증가

확장성 한계

전통적인 Windows 모놀리식 및
온 프레미스 아키텍처는 최대
수요에 대비하기 위해 과도한
리소스 할당이 필요

보안 및 지원 문제

Windows/SQL Server는
수년마다 지원이 종료되어 패치,
업그레이드, 보안 규정 위반
가능성이 증가

혁신의 병목

모놀리식 아키텍처는 배포 주기가
느리고 최신 DevOps, AI, 분석
기능과의 통합이 제한

기술 부채의 문제

10개월 전 작성한 프로그램을 수정하는 것은 어제 작성한 프로그램을 수정하는 것보다 시간이 많이 걸린다. **그 시기의 배경지식**을 다시 떠올리는 것도 필요하고 그동안 새로 나타난 **상호의존성**을 처리하는 것은 그보다 더 많은 시간이 걸릴 수도 있다.

- <일 잘하는 엔지니어의 생각 기법>에서,
캐리 밀샵

23%

낭비되는 개발자 시간

개발자들은 기술 부채로 인해 평균적으로
업무 시간의 23%를 낭비하고 있습니다.

- Journal of Systems and Software



동작한다면 건드리지 마라

- 프로그래머 사이의 불문율

AWS Transform for .NET

대규모로 .NET 애플리케이션을 현대화하는
최초의 Agentic AI 기반 경험



속도

.NET 애플리케이션 변환을
최대 5배 가속화



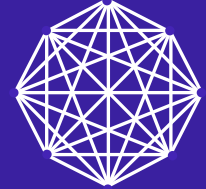
비용 절감

운영 비용을 최대 40% 절감



품질

생성형 AI LLM을
사용하여 .NET 포팅 품질
향상

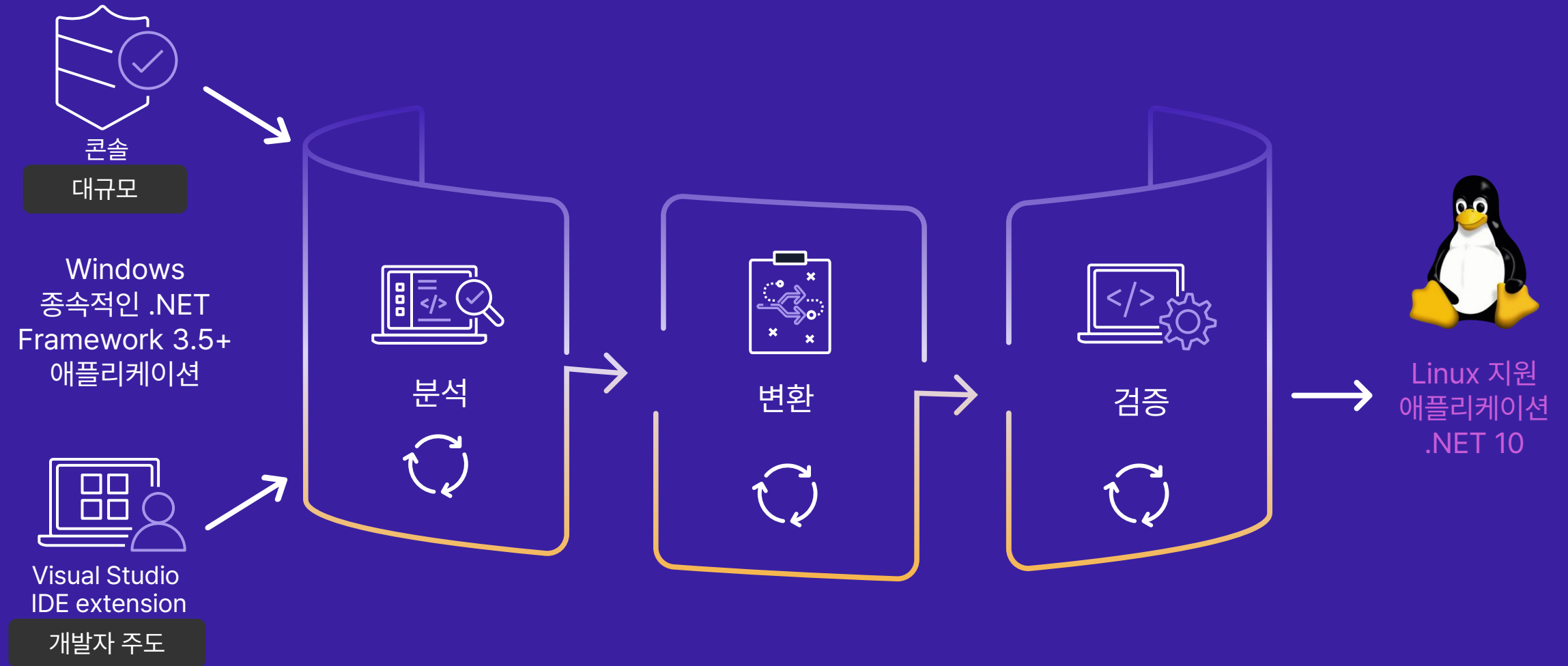


확장성

수백 개의 .NET
애플리케이션을 동시에
포팅하여 현대화 확장

AWS Transform for .NET 변환 에이전트

.NET 애플리케이션을 Linux로 포팅하기 위한 에이전트



지원되는 버전 및 프로젝트 유형

웹 콘솔과 IDE에서 공통 기능

- **.NET Framework 3.5+** 에서 .NET 8 또는 10으로, 클래스 라이브러리를 C#의 .NET Standard로 이전
- **Presentation layer:** MVC Razor, Web Forms를 Blazor, SPA (BLL)로 변환
- **Core layer:** WCF, Web API, 클래스 라이브러리, 콘솔 앱, 단위 테스트 프로젝트 (MSTest, NUnit, XUnit)
- **Data Access layer:** EF 와 ADO.NET*
- **Preview project types:** WinForms 또는 WPF 데스크톱 프로젝트, Xamarin 모바일 프로젝트, VB.NET으로 작성된 프로젝트

* US East (N. Virginia) region 만 가능

이전

Windows 기반 애플리케이션 스택

.NET Framework 3.5+

UI Layer (Web Forms, MVC
Razor)
Business Logic Layer
Data Access Layer

대규모 현대화



웹 콘솔

이후

오픈 소스, 클라우드 네이티브 스택

크로스 플랫폼 .NET 8 & 10

UI Layer (Web Forms→Blazor, MVC Razor)
Business Logic Layer (현대화)
Data Access Layer (현대화)

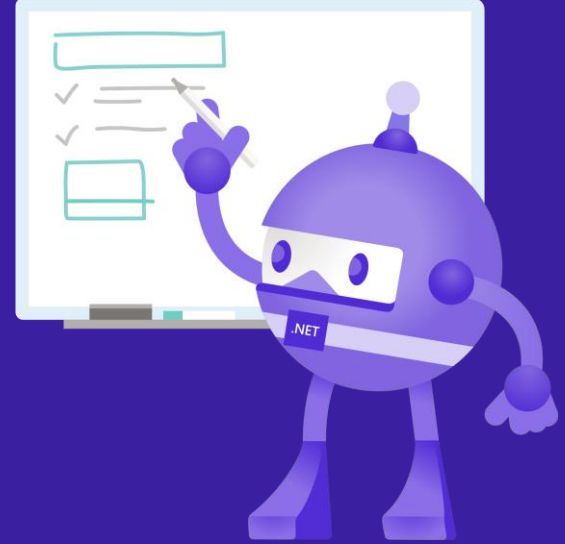
개발자 주도 현대화



Visual Studio IDE extension

IDE와 함께 동작

AWS toolkit for Visual Studio



AI 주도의 개발자 경험



Visual Studio
IDE extension

1. 평가

2. 계획



3. 계획 검증

에이전트 워크플로우가 상황에
맞춰 스스로 계획 및 적응

HITL 지원

인간 개입(HITL) 검증을 통한
신뢰성 보장

6. 다음 단계

4. 변환

5. 변환 검증

다음단계
확인

단계
구현

로그
작성

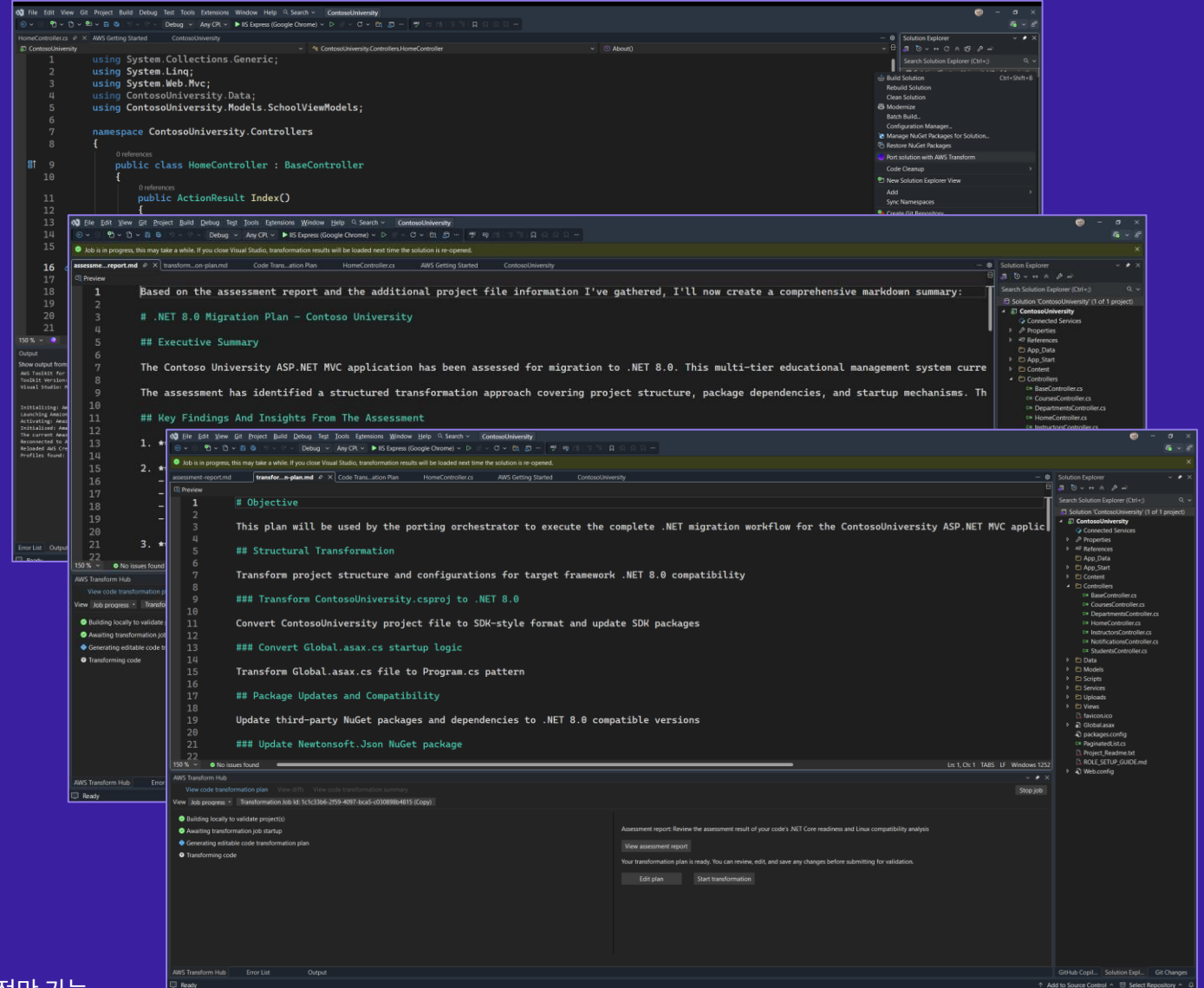
구현
검증

코드
커밋

개발자 경험

IDE에서의 변환

- Visual Studio 2026 or 2022를 이용한 변환작업 – 프로젝트/솔루션을 선택한 후 우클릭하여 변환하고, 웹에서 진행 상황을 확인할 수 있도록 워크스페이스를 생성
- 평가 보고서를 검토하여 발견된 문제점과 현대화 시 해결해야 할 과제를 이해
- 변환 계획을 검토하고 필요에 따라 단계를 추가/삭제하며, 패키지 업데이트 상세 정보와 코딩 패턴 예제*를 지정하여 커스터마이징 작업

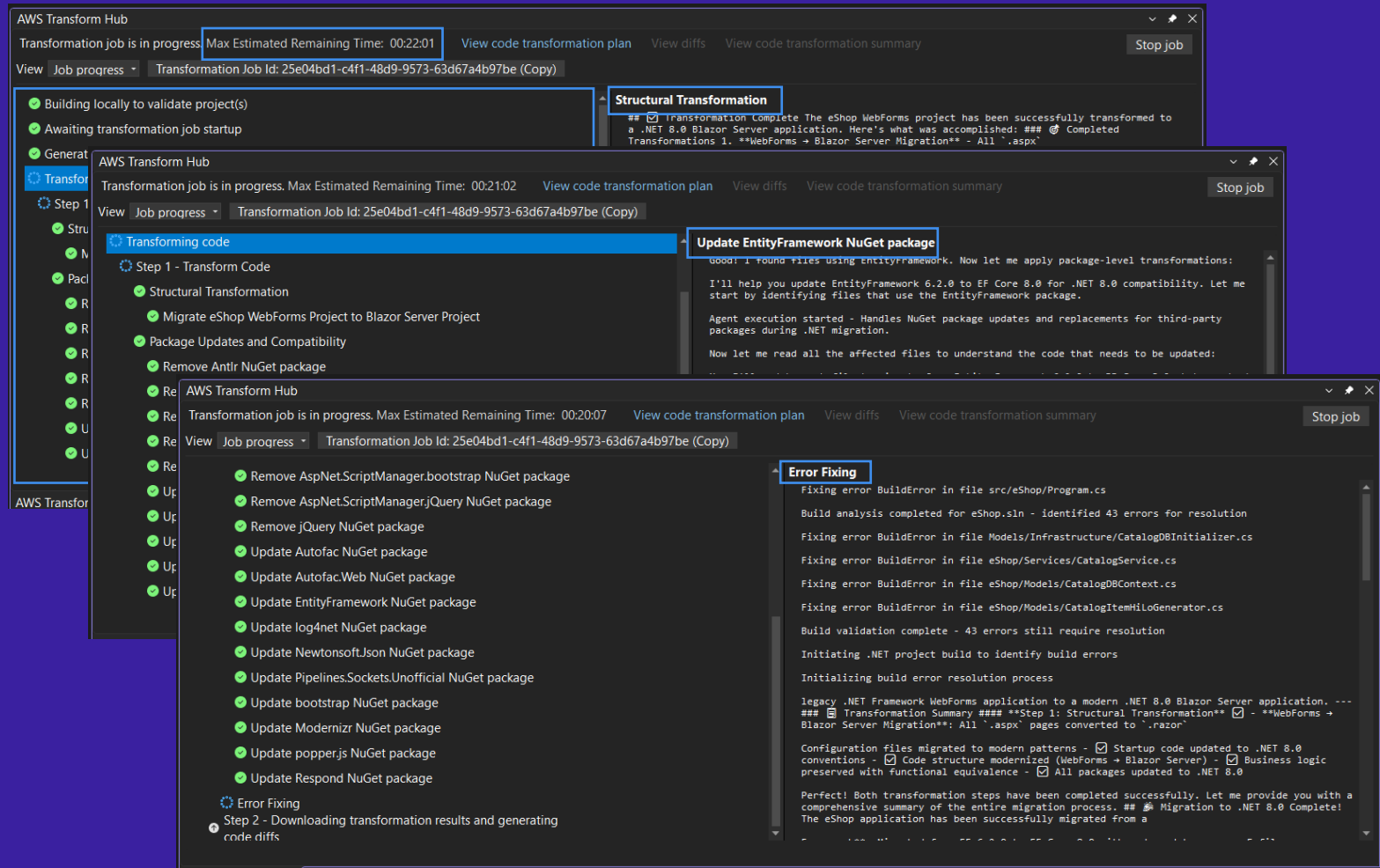


* US East (N. Virginia) 리전만 가능

개발자 경험

변환 작업 모니터링

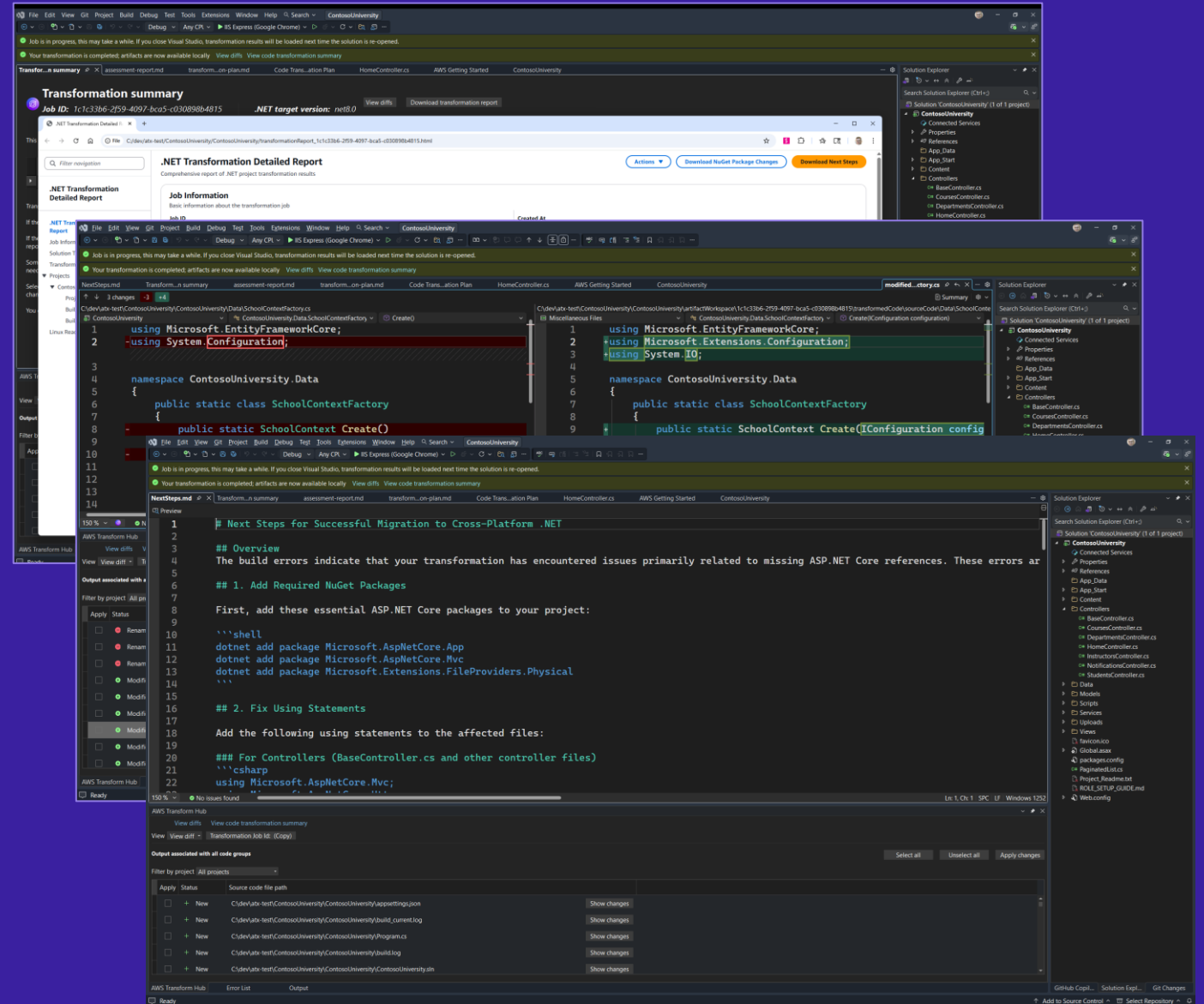
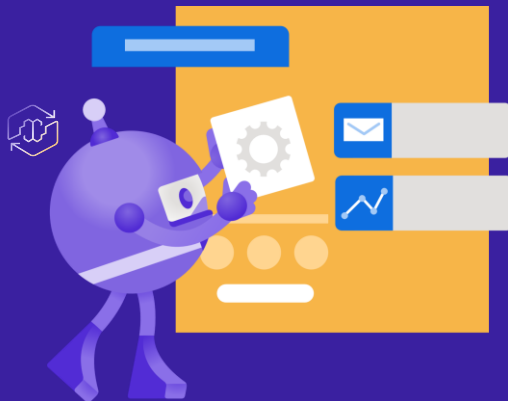
- 변환 단계 및 작업 로그를 통한 실시간 진행 상황 가시성 및 추적
- 완료까지 남은 예상 시간을 표시
- **Headless** 변환방식으로 IDE를 닫고 다른 작업을 할 수 있으며, 완료되면 이메일로 알림 수신 가능

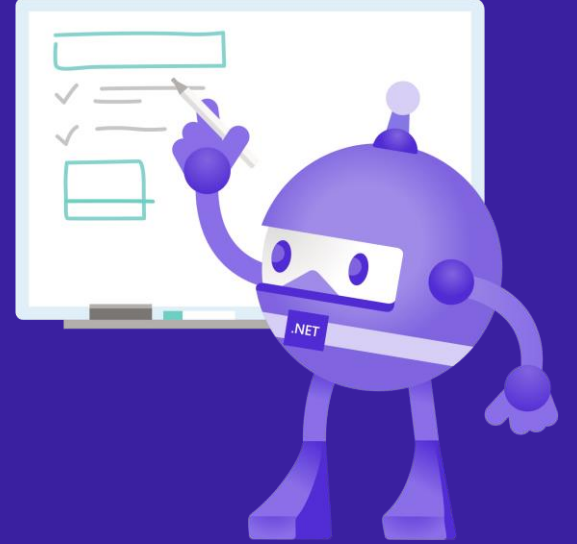


개발자 경험

결과를 확인하고 다음 단계로 진행

- 변환 보고서를 확인하여 무엇이 변경되었고 그 이유, 단위 테스트 결과, 빌드 오류 및 Linux 준비 상태 문제를 파악
- 코드 비교를 통해 변경 사항을 검토하고 솔루션에 적용
- **Next Steps** 마크다운 파일에 남은 포팅 작업을 안내: 수정된 계획으로 변환을 다시 실행하거나 AI 코딩 어시스턴트를 활용하여 추가 작업 진행





Demonstration

AWS Transform for .NET을 이용한 WebForm 애플리케이션을 Blazor로 변환하기

Solution Explorer

Search Solution Explorer (Ctrl+;) 🔍

Solution 'RiskCalculatorWebForm'

- risk-calculator-webapp
 - Connected Services
 - Properties
 - References
 - .vs
 - App_Code
 - App_Data
 - App_Start
 - bin
 - Classes
 - Content
 - Controls
 - obj
 - Scripts
 - Bundle.config
 - CalculateVar.aspx
 - Default.aspx
 - Global.asax
 - MonteCarlo.aspx
 - packages.config
 - Portfolio.aspx
 - RiskAlerts.aspx
 - Site.Master
 - Web.config

Solution Explorer Git Changes

Error List

Entire Solution 0 Errors 1 Warning 0 Messages Build + IntelliSense Search Error List 🔍

Code	Description	Project	File	Line	Suppression State
	Some NuGet packages were installed using a target framework different from the current target framework and may need to be reinstalled. Visit https://docs.nuget.org/docs/risk-calculator-webapp-workflows/reinstalling-packages for more information.			0	

Error List Output

AI 코딩 어시스턴트와 함께 변환 마무리

선택한 코딩 어시스턴트를 사용하여 검토 및 변경

- 대량 변환을 위해 먼저 **AWS Transform** 사용 (비동기, 대규모)
- 남겨진(추가) 작업에 대해서 **Kiro/Amazon Q Developer** 사용 (동기, 대화형)
- 사전 구축된 워크플로우가 특정 요구사항을 충족하지 못할 때 **AWS Transform Custom** 사용



Thank you!

bckang@amazon.com

